```
        Tasks = new List<string>                ❸
        {                                       ❸
            "Get fuel",                         ❸
            "Check oil",                        ❸
            "Check tyre pressure"               ❸
        };                                      ❸
    }
}
```

❶ The PageModel is passed to the Razor view when it executes.

❷ The public properties can be accessed from the Razor view.

❸ Building the required data: this would normally call out to a service or database to load the data.

You can access the `PageModel` instance itself from the Razor view using the `Model` property. For example, to display the `Title` property of the `ToDoItemModel` in the Razor view, you'd use `<h1>@Model.Title</h1>`. This would render the string provided in the `ToDoItemModel.Title` property, producing the `<h1>Tasks for today</h1>` HTML.

TIP Note that the `@model` directive should be at the top of your view, immediately after the `@page` directive, and it has a lowercase `m`. The `Model` property can be accessed anywhere in the view and has an uppercase `M`.

In most cases, using public properties on your `PageModel` is the way to go; it's the standard mechanism for passing data between the page handler and the view. But in some circumstances, properties on your `PageModel` might not be the best fit. This is often the case when you want to pass data between view layouts. You'll see how this works in section 17.4.

A common example is the title of the page. You need to provide a title for every page in your application, so you could create a base class with a `Title` property and make every `PageModel` inherit from it. But that's